The Scry Project

ISA Specification

Emad Maroun Oliver Keszocze

August 20, 2026

Version 0.0.1

Contents

1 Motivation and Introduction **5**
- 1.1 Execution Model . . . 5
  - 1.1.1 Forward-Temporal Referencing . . . 5
  - 1.1.2 First Output of Last Instruction . . . 6
- 1.2 Tagged Data . . . 6

2 Terminology and Technicalities **8**
- 2.1 Terminology . . . 8
- 2.2 Scalar Types . . . 9
  - 2.2.1 Unsigned Integers . . . 9
  - 2.2.2 Signed Integers . . . 9

3 The Stack **10**
- 3.1 Calling Convention . . . 11

4 The Instructions **13**
- 4.1 Instruction Formats . . . 13
- 4.2 Shared Fields . . . 13
- 4.3 Assembly . . . 14
- 4.4 NEXT Instructions . . . 15
  - 4.4.1 Trap . . . 15
  - 4.4.2 No-operation . . . 16
  - 4.4.3 Grow . . . 16
  - 4.4.4 Constant . . . 16
  - 4.4.5 Fence . . . 17
- 4.5 ALU Instructions . . . 17
  - 4.5.1 ALU Reference . . . 18
  - 4.5.2 Number of provided operands . . . 20
  - 4.5.3 Effective input type . . . 20
  - 4.5.4 Operation semantics . . . 20
- 4.6 Encoding Overview . . . 32

5 Assembly **33**
5.1 ALU Instructions 33
5.1.1 Different ALU reference modes 33

1 Motivation and Introduction

Emad will fill this chapter with an inspiring motivation why this is an awesome idea...

1.1 Execution Model

Scry is an architecture built on two foundations: *forward-temporal referencing* (FTR) for moving data between instructions, and *internal tagging* for typing that data. This section introduces the model that the rest of the document assumes.

1.1.1 Forward-Temporal Referencing

Unlike register-based architectures, Scry instructions do not name a storage location for their operands. Instead, an instruction produces one or more outputs, and each output carries a *forward-temporal reference*: a value that specifies *when* the output is consumed rather than *where* it is stored.

A reference is measured in instruction *executions*, counted forward from the producing instruction. A reference of 0 means the output is consumed by the next instruction that executes; a reference of 1 means the instruction after that, and so on. Because references are temporal and not spatial, they follow control flow: whichever instruction actually executes at the referenced count is the consumer. If execution branches, the output flows to whatever instruction is reached, not to a fixed address.

An instruction's inputs are therefore *implicit*. An instruction does not state where its operands come from; it receives whatever outputs preceding instructions have targeted at it. The inputs form an ordered list, the instruction's *ready queue*. Inputs are ordered by the execution order of their producers: the output of the earliest-executed producer is first, the next-earliest second, and so on. When a single instruction produces several outputs aimed at the same consumer, their relative order is defined by that instruction (see the `echo`, `dup`, and ALU definitions). An instruction consumes at most four operands. If more than four outputs are targeted at the same instruction, only the first four in ready-queue order are kept; any beyond the fourth are dropped. Input order is significant — for example, `sub` subtracts its second input from its first — so it is the responsibility of the compiler or programmer to arrange instructions so that operands arrive in the intended order.

The output reference field (`ref`, see section 4.2) is 5 bits wide, giving a reach of 0 to 31 executed instructions. When a value must reach a consumer beyond that reach, or must be

delivered to more than one consumer, the dedicated data-flow instructions `echo` (forward), `echo.l` (forward with extended reach), and `dup` (duplicate) are used to re-target or replicate it. An instruction that needs a value it cannot receive directly is fed by an `echo` chain that keeps the value alive and moves it forward.

Because outputs are consumed rather than stored, no value outlives its reference. A value that has been delivered to its consumer ceases to exist; there is no architectural register file to reason about, and no anti-dependencies between otherwise independent instructions.

Other than the four-operand limit above, the ISA imposes no capacity limits. In particular, there is no architectural bound on the number of simultaneously in-flight values, on the number of pending control-flow triggers, or on call depth. Implementations must behave as if these resources were unbounded, using caches and memory to back their on-chip structures when their capacity is exceeded.

1.1.2 First Output of Last Instruction

Some instructions take an input from the **f**irst **o**utput of the **l**ast **i**nstruction executed (FOLI). This is an explicit deviation from the regular forward referencing rule and ignores the output targets of preceding instructions. Intuitively, all instructions produce a FOLI (which is implicitly their first out in their output order), which is available to the next instruction only. An instruction that takes a FOLI input, but is preceded by an instruction that does not produce one, will implicitly receive a NaR instead. Instructions that do not produce any outputs also do not produce a FOLI for the following instruction.

All instructions that produce at least one output also produce a FOLI. The FOLI is always the instruction's first output in its output order, resolved as follows. Instructions that produce an output to the next instruction count that output as their FOLI. Instructions that produce outputs only via an output reference have their first output as the FOLI. Instructions that produce outputs with multiple output references have their first reference's first output as the FOLI. When an instruction produces both an output to the next instruction and outputs via an output reference, the output to the next instruction takes precedence and is the FOLI. Instructions consuming a FOLI discard any operands in their ready queue.

This has been removed from the intended instructions. Might be added later.

1.2 Tagged Data

Scry is an *internally tagged* architecture: every value carried between instructions is a tagged scalar or a vector of tagged scalars (chapter 2), annotated with the type that describes how its bits are interpreted. These tags are tracked only inside the processor. They are *not* stored in memory alongside the data; a value written to memory is stored as plain bytes, and its type is re-established by the load that reads it back (via the load's type field).

Because values carry their types at run time, Scry instructions are *polymorphic*: an instruction inspects the tags of the operands in its ready queue to select both the operation performed

and the type of its result. For example, a single `mul` performs an unsigned multiplication when its operands are unsigned and a signed multiplication when they are signed, and its result type follows the effective-input-type rules of section 4.5.

A value acquires its tag in one of three ways:

1. A typed instruction sets the tag together with the value. Loads (`ld`, `ld.s`) and `const` carry an explicit type field that becomes the tag of the produced value.
2. An instruction derives the result tag from the tags of its inputs, as most ALU instructions do.
3. An instruction explicitly changes the tag. The `cast` instruction re-tags its input to a given type.

The type combinations an instruction accepts are defined per instruction. Combinations that an instruction does not define are type conflicts; how a type conflict is reported is implementation specific, but it must not silently produce an unspecified value (see also the poison values `NaN` and `NaR` in chapter 2).

2 Terminology and Technicalities

2.1 Terminology

Byte A collection of 8 binary bits. This is the smallest unit of data.

Scalar The basic indivisible unit of data. Has an associated type that dictates how many bytes comprise it and the semantics of its bits. E.g. `u8` is a 1-byte (8 bits) unsigned integer. `i32` is a 4-byte (32-bit) signed integer. All scalars of any type can also be the distinct values of `NaN` and `NaR` in addition to any values that comes from reading the bits.

Nan *Not-a-number*. Signals the *intentional* absence of a value, in contrast to `NaR` which signals an unintentional one. It is used where a computation deliberately has no meaningful result but is not an error. Is poisonous. Unlike `NaR` it carries no payload, and it is weaker than `NaR` (see **Poisonous**).

Nar *Not-a-result*. Signals the unintentional absence of a value. Generally generated when an error occurred. E.g. a load from an invalid address produces a `NaR`. Accompanied by a payload that can be used to find the source of the error. Payload is implementation specific. Is poisonous.

Poisonous A scalar with the `NaN` or `NaR` values propagate their *Nanness* or *Narness* to any outputs that depend on them. E.g. adding a value and a Nan will produce a Nan. Value and `NaR` will produce `NaR` (with the same payload as the input `NaR`). `NaR`s are stronger, so adding a `NaN` and `NaR` produces a `NaR`.

Scale The number bytes comprising a scalar of a given type.

Vector A collection of scalars. Vector support is provisional: the behavior of instructions on vector operands (for example element-wise ALU operations and out-of-range indexing in `pick`) is not yet fully specified and is subject to change. Where this document describes vector behavior, it should be read as indicative rather than normative.

Length The number of scalars in a vector.

Size The number of bytes in a vector. I.e. the scale of its scalars times the length. The size of a single scalar is its scale.

usize The native pointer size of a processor implementation in bytes. This is processor specific, with the only restriction being that it must be a power of 2. E.g., a processor with a 32-bit native pointer size has `usize`=4.

Tagged Scalar A scalar value that is annotated (i.e. *tagged*) with its type, e.g. `i16`. In this context, the type is also referred to as the *Tag*. A tagged scalar with tag t and value v is written $\langle t, v \rangle$.

Tag See **Tagged Scalar**

2.2 Scalar Types

All scalars must have a scale that is a power of 2 (1,2,4,.. bytes). The currently supported types are specified below.

Naming convention. Integer types are named by their width in bits: `uN` is an `N`-bit unsigned integer and `iN` an `N`-bit signed integer (for example `u8`, `u16`, `i32`). The width `N` in bits is always eight times the scale in bytes, so `u8` occupies one byte, `u16` two bytes, and so on.

2.2.1 Unsigned Integers

Unsigned binary integers `uN` where `N` is the width in bits (`u8`, `u16`, ...).

Examples

Scalar Type	Bits	Value
`u8`	`00001001`	9
`u16`	`00001001 00001001`	2313
`u32`	`10000000 00000000 00000000 00000000`	2147483648

2.2.2 Signed Integers

Signed, two's complement, binary integers `iN` where `N` is the width in bits (`i8`, `i16`, ...).

Examples

Scalar Type	Bits	Value
`i8`	`00001001`	9
`i8`	`10001001`	-119
`i16`	`00001001 00001001`	2313
`i16`	`10001001 00001001`	-30455
`i32`	`10000000 00000000 00000000 00000000`	-2147483648

3 The Stack

The stack can be used to store a function's temporary data, and to pass data between function calls that cannot be passed as an argument to the call instructions. The stack is a contiguous piece of memory. The load-stack instruction can be used to load directly from the stack, which allows an offset to be used to index into the stack without using specific memory locations.

A function may start its life with an empty stack frame. It can then allocate more space on its stack frame. The frame is split into two parts, the "private" frame and the "shared" frame. From the perspective of a load-stack, the private frame comes first (is indexed by lower values.)

Formally, a frame is described by three boundaries in the frame's *offset space* — the space that load-stack indexes into, with offset 0 at the base of the private frame — ordered $base \leq split \leq top$. The private frame is the offset range $[base, split)$ and the shared frame is the offset range $[split, top)$. An empty frame has $base = split = top$. The reserve and free instructions move $split$ and top; $base$ is fixed for the lifetime of the frame. The instruction's `t` bit selects which frame the operation targets (`t=1`: private, `t=0`: shared). How this offset space maps onto real memory addresses — in particular whether growing the frame moves toward higher or lower addresses — is not defined by the ISA and is left to the implementation. In the descriptions below, "increase" and "decrease" refer to the offset space, where the shared frame lies above the private frame.

Reserve shared (`t=0`) top increases by the requested amount. The new bytes are fresh, uninitialized memory and become the top of the shared frame. This is the only way the total frame size grows without moving data.

Reserve private (`t=1`) $split$ increases by the requested amount, so the private frame grows and the shared frame shrinks by the same amount — data is effectively moved from the shared frame into the private frame. If the shared frame holds fewer bytes than requested, top is raised so that $top = split$, and the shortfall is fresh, uninitialized memory that now lies in the private frame.

Free private (`t=1`) $split$ decreases by the requested amount. The freed bytes rejoin the shared frame.

Free shared (`t=0`) top decreases by the requested amount, shrinking the shared frame. If this would move top below $split$ (i.e. the shared frame is smaller than the amount freed), $split$ is lowered to top as well, so the private frame shrinks by the remainder.

When a call is executed, the shared frame of the caller becomes the private frame of the callee. When a return is performed, the private frame of the callee becomes the shared frame of the caller. This is how arguments and results that do not travel as instruction operands are handed across a call: the caller places them in its shared frame, and they appear in the callee's private frame (and vice versa on return).

Reserving memory may fail, if there is no more memory left. A trap will be thrown. How and when this happens in implementation dependent.

Do we need to specify is "reserving" means new memory is in high/lower addresses? Could leave as implementation defined. Simulator currently reserves using higher addresses.

3.1 Calling Convention

A call can pass at most four arguments as instruction operands: all values whose references resolve at the call trigger are delivered to the first instruction executed in the callee (see the `call` instruction), and an instruction consumes at most four operands (subsection 1.1.1). A call cannot spread delivery over several consumer instructions either: a value's flight time either resolves exactly at the call trigger, making it an argument, or it is greater and pauses for the duration of the call, resuming in the caller after the return. No reference can therefore target the callee's second or later instructions, and an argument beyond the fourth would be dropped by the queue-capacity rule. The same holds symmetrically for values returned through the return trigger.

By convention, values beyond the fourth are therefore passed on the stack. The first four arguments are passed as operands of the call trigger, in argument order, and the first four return values as operands of the return trigger, in order. For any further values, the *caller* reserves a single area on its shared frame before issuing the call; the call turns this area into the base of the callee's private frame. The area is laid out as follows, in the callee's offset space (offset 0 at the area's start):

- The *return area* begins at offset 0 and holds the return values beyond the fourth, in order, the fifth first: each is placed directly after the previous one, at the next offset aligned to its scale. Let R be the total size of this layout ($R = 0$ if there are at most four return values).

- The *argument area* begins at offset R rounded up to the 16-byte frame alignment and holds the arguments beyond the fourth, laid out the same way, the fifth argument first. Note that the position of the argument area therefore depends on the callee's *return* signature as well.

The caller writes the stack-passed arguments into the argument area before issuing the call. The callee reads them from there and writes its stack-passed return values into the return area at any point before its return trigger; the two areas are disjoint, so the arguments remain readable for the callee's entire body.

The area belongs to the callee for the duration of the call — it is simply the start of its private frame: the callee may read or overwrite any of it, and its contents after the return are unspecified except for the return area.

Frame responsibility follows one rule: *whoever reserves an area frees that same area*. The caller reserves the argument-and-return area before the call and, after reading the return values, frees the same amount from the shared frame it receives back. The callee reserves whatever private frame it needs for its own data above the area and frees exactly that before returning, so the frame it hands back is the area the caller reserved.

The offsets above are aligned in the frame's offset space; whether they are also aligned in real memory depends on the memory alignment of the frame base, which is currently implementation defined (see the alignment discussion under the stack-control instructions).

4 The Instructions

4.1 Instruction Formats

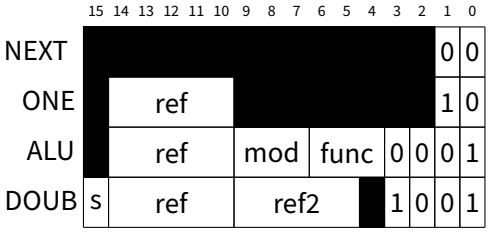
We have four instruction formats. The formats are groups of instructions that have the same output behavior and shared field positions:

NEXT These instructions output any number (including zero) of values to the next instructions.

ONE These instructions output to the instruction at the given offset.

ALU ALU instructions have their own output format that is described its own section.

DOUB These instructions output two instructions, each at its given offset.



All instructions are 2 bytes (16-bit). Scry uses little endian, meaning least significant bytes are stored at the lower addresses in memory. This is the case for both instructions and data. Gray fields in the instruction encoding are unused for now. This should be assumed to be "0", but may change in the future.

4.2 Shared Fields

Some fields may appear in multiple instructions with the same (shared) meaning to their bits. These fields are specified here:

- ty: The short type specifier using 3 bits. Specifies the following integer types:

Encoding	Type
000	u8
010	u16
100	u32
110	u64
001	i8
011	i16
101	i32
111	i64

- type: The type specifier using 4 bits. Specifies the following types. The low bit selects signedness (0: unsigned, 1: signed) and the upper three bits select the scale as a power of two (000: 1 byte, 001: 2 bytes, …, 100: 16 bytes).

Encoding	Type
0000	u8
0010	u16
0100	u32
0110	u64
1000	u128
0001	i8
0011	i16
0101	i32
0111	i64
1001	i128

  The remaining encodings are reserved and not yet supported.

- ref: Output reference of 5 bits. This is a forward-temporal reference (see the execution model, subsection 1.1.1): the value r means the produced output is consumed by the $(r+1)$-th instruction executed after the producer, i.e. $r = 0$ targets the next executed instruction. If a second output reference exists, it is named "ref2".

- refl: Output reference of 10 bits, interpreted like `ref` but with a reach of 0 to 1023.

4.3 Assembly

We will map the instruction encoding to an assembly language. In assembly, each instruction start with a mnemonic followed by arguments. The mnemonics are not unique for each instruction. To map an assembly instruction, the mnemonic and arguments must be parsed in totality to determine which instruction it represents.

In the assembly declaration, curly-brackets ("") encapsulate rules for encoding/interpretation. Anything not within the brackets is literal and must be present in the instruction.

Mapping: `{ARG : FLD}`

All rules are mappings from the assembly argument (ARG) to the encoded field (FLD). In its simplest form, a literal value may be mapped into the field: `{00:xx}`. Here the literal value 00 is mapped to the field `xx`, which must match the bit-width of the literal. Complex argument can also be used, each of which has its own meaning, which can be see below. Shared among them are that they can be converted to an encoded value of the same bit-width as the field.

Multiple fields may be mapped by a single argument. In this case FLD may be split by a "<>" to connected two field, with the first field encoded to the higher-order bits and the second the lower-order bits. E.g., `{00111:xx<>yyy}` will encode 00 in `xx` and 111 in `yyy`.

Complex arguments:

- Dec: Decimal number encoded directly
- DecPow2: Decimal number that is a power of two. Its power of 2 index is encoded. I.e., 1→0, 2→1, 4→2, etc.
- Type: Short of long type specifier
- mnem: Special case for ALU. The whole mapping must be replaced by an ALU instruction mnemonic, while the encoded depends on the combination of mnemonic and all other arguments.
- Ref: Output reference. Either as an arrow followed by a literal (in which case the literal is encoded directly), or a reference path.
- Label: Instruction offset to a label. Can either be a literal (in which case it is encoded directly), or it can be a label identifier which must be converted to a relative offset from the current instruction to the label position. The offset must be encoded divided by 2.

Conditional mapping: `{{CND}?  MPT : MPF}`

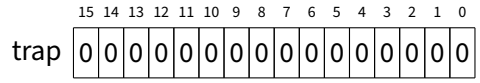
A conditional mapping will map depending on the presence of an argument. The argument is given in CND. If present, the MPT mapping will be applied, otherwise the MPF mapping.

4.4 NEXT Instructions

4.4.1 Trap

Immediately calls the default trap handler. This is primarily to catch errors by always causing an exception if uninitialized data is attempted to be executed. Also makes it possible to put protection around code that should not overflow.

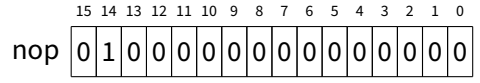
How the default trap handler is identified is implementation specific.




```
trap
```

4.4.2 No-operation

Discards all inputs and produces no outputs. Any `NaR`s or `NaN`s in the inputs are ignored and do *not* continue to be poisonous.

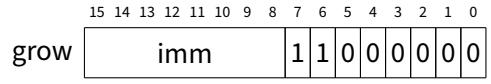



```
nop
```

4.4.3 Grow

Shifts the first input left by 8 bits and then inserts `imm` into the lower 8 bits. Outputs to the next instruction. The scale (and type) of the output is the same as that of the input; `grow` does not widen its operand. Any bits shifted beyond the input's scale are therefore discarded. Any other inputs are passed on to the next instruction.

Because `grow` preserves the scale, building a constant wider than a single `const` immediate requires the value to already have sufficient scale: emit a `const` of the target type (whose immediate occupies the low 8 bits) and then `grow` it one byte at a time. For scales larger than the `const` type can express (e.g. 128-bit values), `cast` the value to the wider type before growing it.




```
grow {Dec:imm}
```

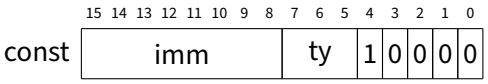
4.4.4 Constant

Produces one immediate integer scalar of the given type output to the next instruction.

If any inputs are given, they are forwarded to the next instruction following the immediate. This allows flexibility in how to order the immediate. Assuming the consumer is directly

following this instruction, if the immediate needs to be after other inputs to the next instruction, the other inputs are simply sent directly to that instruction. If the immediate is needed as the first, other inputs can be sent to this instruction instead, which will forward them following the immediate in the order.

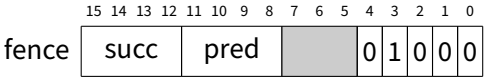
Together with the grow instruction, this is used to construct arbitrary scale constants of any value using $2\times$ the instructions bits as value bits. If other types than those native to this instruction are needed, they should be cast after their bits are set, or cast before using grow.




```
const {Type:ty}, {Dec:imm}
```

4.4.5 Fence

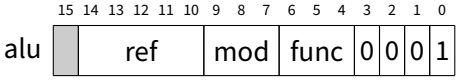
Orders memory accesses across the fence, with the same semantics as the `RISC-V FENCE` instruction. The instruction carries a predecessor set `pred` and a successor set `succ`, each 4 bits. As in `RISC-V`, each set's four bits select access classes — device input (I), device output (O), memory read (R), and memory write (W), from the high bit to the low bit. The fence guarantees that the accesses in the classes named by `pred`, issued by instructions executed before the fence, are ordered before the accesses in the classes named by `succ`, issued by instructions executed after it. The 3-bit sub-class field is reserved for fence-mode variants and is currently assumed to be 0. Any inputs are discarded and no output is produced.




```
fence {Dec:pred}, {Dec:succ}
```

4.5 ALU Instructions

All ALU instructions use the same instruction format. The `mod` and `func` fields, combined, are used to specify distinct operations (addition, subtractions, etc.).



```
{mnem:func<>mod} {Ref:ref}                  // Single output
{mnem:func<>mod} Low, High {Ref:ref}        // Double output, LowFirst
{mnem:func<>mod} High, Low {Ref:ref}        // Double output, HighFirst
{mnem:func<>mod} High {Ref:ref}, Low =>     // Double output, LowNext
{mnem:func<>mod} Low {Ref:ref}, High =>     // Double output, HighNext
{mnem:func<>mod} Low {Ref:ref}              // Double output, LowOnly
{mnem:func<>mod} High {Ref:ref}             // Double output, HighOnly
```

4.5.1 ALU Reference

For mod=111 or mod=000, the operation produces 1 output to the instruction at the output reference. When mod≠111 and mod≠000, it means the operation produces 2 outputs. The outputs are named “Low” and “High” and are defined by each instruction.

The 6 values of mod define how the outputs are produced:

001 *LowFirst*: Both outputs have the same target according to the output reference argument, with the low being first in the order.

010 *HighFirst*: Both outputs have the same target according to the output reference argument, with the high being first in the order.

011 *HighNext*: The low follows the output field, while the high is passed to the next instruction.

100 *LowNext*: The high follows the output field, while the low is passed to the next instruction.

101 *LowOnly*: The low follows the output field, while the high is discarded.

110 *HighOnly*: The high follows the output field, while the low is discarded.

The `mod` field is thus overloaded. For a given `func`, the two values `mod` $= 000$ and `mod` $= 111$ each select a *distinct single-output operation*, while the remaining six values select a *two-output operation* together with one of the six output routings above. This has two consequences that Table 4.1 reflects. First, the single-output operations (`eq`, `lt`, `gt`, `add.s`, `sub.s`, `and`, `or`, `xor`, `isnar`, `narto`) have no output-routing choice; they always produce their one output to the output reference. Second, the two-output operations (`add`, `sub`, `shift`, `mul`, `div`) cannot be encoded with `mod` $= 000$ or `mod` $= 111$; to obtain a single result from them (discarding the other output) the *LowOnly* or *HighOnly* routing is used.

Table 4.1 defines each ALU instructions. All currently defined instructions need at least one operand. Most discard any operand after the second; the exception is `isnar`, which accepts up to four operands.

Do we need rotates?

Table 4.1: ALU instruction encodings. Dashes signify unused encodings.

func	mod	Mnemonic	Operation	Implicit
000	000	`eq`	Equal comparison	0
	111	`add.s`	Addition, saturating	1
	mod	`add`	Addition, wrapping (Low), with carry (High)	1
001	000	`and`	Bitwise AND	1
	111	`sub.s`	Subtraction, saturating	1
	mod	`sub`	Subtraction, wrapping (Low), with carry (High)	1
010	000	`lt`	Less-than comparison	0
	111	`gt`	Greater-than comparison	0
	mod	`sh`	Shift left/right by signed amount (Low) with carry (High)	1
011	000	`or`	Bitwise OR	All bits set
	111	`xor`	Bitwise XOR	All bits set
	mod	-	-	-
100	000	`isnar`	Whether inputs include NaR	-
	111	`narto`	Extract NaR payload	-
	mod	`mul`	Full multiplication, least significant bits (Low), most significant bits (High)	`usize`
101	000	-	-	-
	111	-	-	-
	mod	`div`	Euclidean Division (low) with remainder (High)	2
110	-	-	-	-
111	-	-	-	-

	u8	u16	u32	u64	i8	i16	i32	i64
u8	u8	u16	u32	u64	u8	u16	u32	u64
u16	u16	u16	u32	u64	u16	u16	u32	u64
u32	u32	u32	u32	u64	u32	u32	u32	u64
u64	u64	u64	u64	u64	u64	u64	u64	u64
i8	u8	u16	u32	u64	i8	i16	i32	i64
i16	u16	u16	u32	u64	i16	i16	i32	i64
i32	u32	u32	u32	u64	i32	i32	i32	i64
i64	u64	u64	u64	u64	i64	i64	i64	i64

Table 4.2: Effective input type for the default types.

4.5.2 Number of provided operands

Operations that intrinsically require two operands (addition, subtraction, multiplication) accept single operands as well. In this case, an implicit default operand is used as the second. The implicit operand is chosen as the one most likely to be used. It can be seen from Table 4.1's last column. It has the same type as the single input operand.

If no inputs are provided to the operations, they will all produce a `NaR` as a result.

4.5.3 Effective input type

When the input types of all consumed operands are the same, the effective operation type is the same as the input types. If the input types are different, operand types are converted using the following rules.

1. If one of the inputs is unsigned, they both become unsigned.
2. The input of the smaller scale is zero/sign extended (based on its original type) to the scale of the other input.

The effective input type (`eit`) is the result of applying the above rules. The `eit` values for the default data types are shown in Table 4.2.

4.5.4 Operation semantics

eq, lt, gt

Checks whether the two operands have the same logical value (eq), less-than logical value (lt), or greater-than logical value (gt). Does **not** use `eit`. Uses logical values and not bit-pattern comparisons. I.e., two `u8`s must have the same bit pattern to be equal, while a `u8` and an `i8` cannot be equal if the `i8` is negative. Likewise, a negative `iN` is always less than any `uN`.

Output: `u8` of value 0 for logical false and 1 for logical true.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
eq	2	0	`u8`	mod: 0 0 0 func: 0 0 0
lt	2	0	`u8`	mod: 0 0 0 func: 0 1 0
gt	2	0	`u8`	mod: 1 1 1 func: 0 1 0

add.s, sub.s

Saturating addition/subtraction using effective input types. For subtraction, the second input is subtracted from the first.

Output: The saturated addition/subtraction result of the same type as the effective input type.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
add.s/add	2	1	`eit`	mod: 1 1 1 func: 0 0 0
sub.s/sub	2	1	`eit`	mod: 1 1 1 func: 0 0 1

add, sub

Wrapping addition/subtraction using effective input types. For subtraction, the second input is subtracted from the first.

Output: The low output is the wrapping addition/subtraction result of the same type as the effective input type. The high output is an u8 of 0 if no wrapping occurred and 1 if wrapping occurred.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
add	2	1	$\langle$`eit`, `u8`$\rangle$	mod: mod func: 0 0 0
sub	2	1	$\langle$`eit`, `u8`$\rangle$	mod: mod func: 0 0 1

and, or, xor

Bitwise AND/OR/XOR operations on effective input types.

Output: The result as the same type as the effective input type.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
and	2	1	`eit`	mod: 000 func: 001
or	2	All bits set	`eit`	mod: 000 func: 011
xor	2	All bits set	`eit`	mod: 111 func: 011

shift

Unified left/right shift with carry. Does **not** use effective input types. The first operand is the value to shift, the second is the shift amount. The direction follows the amount's logical value: a positive amount shifts left, a negative amount shifts right by the amount's magnitude, and an amount of zero leaves the value unchanged. If the first operand is signed, right shifts are arithmetic, otherwise logical.

Conceptually, the first operand sits in an unbounded string of bits whose other positions hold zero — except that for a signed first operand, the positions above it hold copies of its sign bit — and the whole string is shifted by the amount. The low output is the window of the string at the operand's original position; the high output is the adjacent window in the direction the bits exit: directly above the operand's position for left shifts, directly below for right shifts. Both outputs have the type of the first operand.

In the table below, t_1 denotes the type of the first operand (shifts do not use `eit`).

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
sh	2	1	$\langle t_1, t_1 \rangle$	mod: mod func: 010

mul

Full multiplication using effective input types. The two operands are multiplied to a double-width product.

Output: Low output containing the least-significant half of the multiplication result of the same type as the effective input type. High output containing the most-significant half of the multiplication result of the same type as the effective input type.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
mul	2	usize	$\langle \texttt{eit}, \texttt{eit} \rangle$	mod: mod func: 100

Should the high output be strictly unsigned?

div

Euclidean division with remainder using effective input types.

Output: Low output containing the division result of the same type as the effective input type. High output containing the remainder of the division has the same type as the low output but unsigned (euclidean division always has non-negative remainder).

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
div	2	usize	$\langle \texttt{eit}, Unsigned(\texttt{eit}) \rangle$	mod: mod func: 1 0 1

isnar

Checks for the presence of NaRs. Accepts up to 4 inputs.

Output: u8 of value 0 if none of the inputs are NaR, and 1 if any of them are NaR.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
isnar	1-4	-	u8	mod: 0 0 0 func: 1 0 0

narto

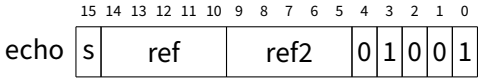
Extract the payload of the first input NaR. If the first input is not a NaR, produces a NaR.

Converts a NaR into an unsigned integer usize bytes wide (i.e. $\texttt{u}\langle 8 \cdot \texttt{usize} \rangle$) holding the NaR's payload. The payload is implementation-specific.

Mnemonic	# Inputs	Implicit	Output type	Bit Pattern
narto	1	-	$\texttt{u}\langle 8 \cdot \texttt{usize} \rangle$	mod: 1 1 1 func: 1 0 0

- Echo – Forward the first two inputs.

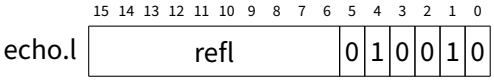
  Forwards the first two inputs with each its output reference. If their offset is the same, their order swaps so the first input to this instruction is the second input to the target instruction. If s=0, all other inputs are discarded. If s=1, all other inputs are forwarded to the next instruction. If the target of the first two inputs is the next instruction, and s=1, the remaining inputs come after the first two.




```
echo {Ref:ref}, {Ref:ref2} {{, =>}? {1:s} : {0:s}}
```

  Echo long – Forward inputs with a high offset.

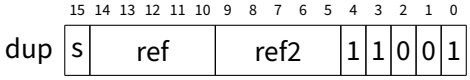
  All inputs are forwarded.




```
echo {Ref:refl}
```

- Duplicate – Duplicates inputs.

  Outputs the given inputs at least twice, with the given offsets. If s=1, the inputs are duplicated thrice, with the third copy going to the next instruction. If s=0, no outputs are given to the next instruction.




```
dup {Ref:ref}, {Ref:ref2} {{, =>}? {1:s} : {0:s}}
```

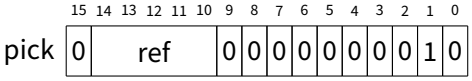
- Pick – Choose a value from inputs.

  Chooses a value from the inputs and outputs it with an offset.

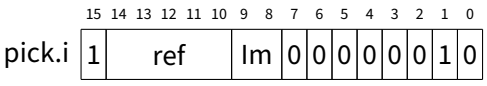
The first input is a condition. If the immediate variant is used, the condition is the immediate. The semantics of this instruction depend on the number of inputs given after the condition:

- 1 Input: Extracts a value from the input (which is treated as a vector) by indexing. The condition is the index. If the condition is longer than 1, each element in the condition is treated as an index. The result is then a vector of the same length as the condition. If the index is out of bounds, produces Nar.
- 2 Inputs: The condition is treated as a boolean. If it is zero, then the first input is chosen. Otherwise the second input is chosen. If the condition is longer than 1, the inputs are also treated as vectors. Each element in the vectors is then treated as scalars would be. The result is a vector of the same length as the condition.
- >2 Inputs: The condition is treated as a switch index. If it is zero, then the first input is chosen. If it is 2, the third is chosen. If it is 3, the fourth is chosen. Like the previous, if the condition is longer than 1, each index is treated as scalar. For the immediate variant, only the two lower order bits are used as index. The rest are ignored.

Define what happens when negative or non-integer?




```
pick {Ref:ref}
```




```
pick {Dec:Im}, {Ref:ref}
```

Load

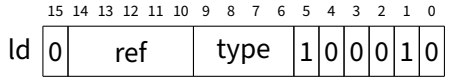
Load a piece of data of the given type.

If the first input is an unsigned integer, it used directly as an address to load from. If the input is a signed integer, it is used as an offset to the address of the load instruction itself, to determine where to load from.

If a second input is given that is a signed integer, that value will be added to the previously determined destination before the load is performed. If it is unsigned, it is treated as an index, scaled to the size of the scalar value and added to the destination before the load is performed.

These two versions allow both for struct loads (where the base is the structs address and the second input is the field offset) and indexed loads (where the base address is the array addres and the second input is the index).

Address operands have no width requirement: an operand narrower than `usize` is zero-extended to `usize`, and one wider than `usize` is truncated to `usize`, before being used in the address computation.




```
ld {Type:type}, {Ref:ref}
```

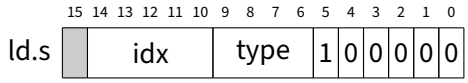
Load Stack

Load a piece of data of the given type from the stack.

'Idx' is the stack index to load from. The address loaded (relative to the base of the stack frame) is $scale \times idx$, where 'scale' is the scale of the item loaded. The effective address is always aligned to the size of the scalar. The 0-index is therefore the next alignment point higher than the base of the stack.

If an input is given as an unsigned integer, it is used as an offset to the stack frame base before the indexing is added. If it is signed it is likewise an offset, but to the secondary stack frame base (negative values necessarily target the primary stack frame.)

Second input behave similarly to non-stack loads.



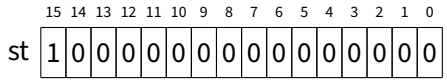

```
ld {Type:type}[{Dec:idx}]
```

Store

Store data

The first input is the value to be stored.

The address to be stored to is calculated similarly to the load instruction. Except additional inputs are given after the first. The scale used with indexing is provided by the stored value.



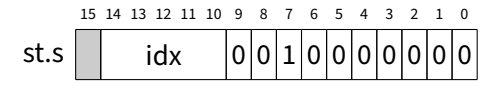
```
st
```

Store stack

Store data on the stack.

The first input is the value to be stored.

The address to be stored to is calculated similarly to the load stack instruction. Except additional inputs are given after the first. The scale used with indexing is provided by the stored value.




```
st [{Dec:idx}]
```

Control Flow Instructions

For every control flow instruction, two addresses are considered: *target address* and *trigger address*. The target address is as usual the address at which execution should continue following the triggering of the control flow. E.g. this could be the address of a function or basic block's first instruction. Unlike other ISAs, control flow in Scry doesn't necessarily trigger after a control flow instruction is executed. Instead, instructions *issue* control flow that should *trigger* following the execution of the instruction at a given address, the trigger address. Conceptually, control flow can be said to trigger between the executions of two instructions. The trigger address is then the address of the instruction immediately preceding the triggering of control flow.

For all control flow instructions an immediate value is provided which is doubled and then added to the address of the instruction to give the trigger address. If the immediate is 0, the trigger address is the address of the instruction, meaning the control flow triggers immediately following the control flow instruction itself. If the immediate is 1, the trigger address is the address of the instruction following the control flow instruction.

It is possible that between a control flow instruction and its trigger another control flow instruction is executed. It's assumed that execution will eventually reach any issued control flow trigger unless a return is triggered. It is also assumed that of all issued control flow, the one with the lowest address is triggered next. This means, jumping past an issued trigger by a new control flow instruction is undefined behavior. Calls and returns don't count, as they return to before the trigger.

Several control flow instructions may issue control flow for the same trigger address. In that case the most recently issued one takes effect and the previously issued ones are discarded;

only one control flow ever triggers at a given trigger address. This gives the instructions sharing a trigger address a last-wins priority order, which lets a multi-way branch be expressed as a run of jumps that share one trigger address: the least specific case is issued first (e.g. an unconditional jump to a default target) and each later, more specific case overrides it. Because all of them trigger at the same address, the same number of instructions is executed on every path, so in-flight values arrive at the target with the same remaining flight time whichever jump wins. The hardware remembers control flow triggered by callers. Returns will cancel all issued triggers in the returning function (callee).

Because output references are temporal (subsection 1.1.1), they interact with control flow as follows. An in-flight value survives a branch trigger, keeping its remaining flight time: the count that determines its consumer simply continues into the instructions executed at the target address. A value is therefore delivered to whichever instruction executes at its referenced count on the taken path, not to a fixed address, and no value is lost merely because a branch was taken.

When control crosses a call boundary, the remaining flight time of the caller's in-flight values stops ticking for the duration of the call and resumes once the call returns (to just after the trigger address). In effect, calls have no bearing on the in-flight values of the caller other than to count as 1 instruction execution for the call instruction itself and 1 execution for the call trigger. I.e. for a value to cross a call boundary and arrive at an instruction following the return, it needs a reference 1 higher than if no call occurred. A callee thus never receives the caller's unrelated in-flight values, and its own in-flight values are private to it. The one exception is values that are passed as arguments to the callee: those are delivered to the first instruction executed in the callee (see the `call` instruction).

- Jump – Jump, optionally based on a condition.

  There are three Jump instruction variants, depending on the number of input operands:

  - If no operands are given, it is an unconditional jump.

    `trig` is the trigger offset. `Imm` is used to calculate the target address. If `Imm` is zero or negative, it is doubled and added to the instruction's address to get the target address. I.e. this will jump "backwards" (or stay on the same instruction if `Imm`=0.) If `Imm` is positive, it is doubled, incremented, and added to the trigger address. I.e. this jumps past the trigger address. Notice that it is impossible to jump to an address between an instruction and its trigger (on purpose).

  - If one operand is given, that is the condition.

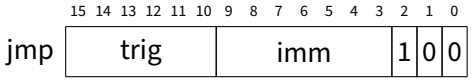
    If `Imm` is zero or negative, the jump is taken if the operand is non-zero. If `Imm` is positive, the jump is taken if the operand is zero.

    This functionality is designed to support both normal branching and looping using the same instruction. Normal branches usually depend on some sort of comparison and usually jump forward in memory if the condition is false. By having forward

jumps happen on zeros, we ensure this functionality mimics the tradition. Loops on the other hand often either follow a counter or stop looping when a condition becomes false and usually jump back. By having back-jumps be taken on non-zeros, this ensures that we can use down-counting effectively and also supports while-loops, that stop as soon as the condition becomes false (which is the same as zero.)

- If two operands are given, the first is the condition, the second is used to calculate the address, while `Imm` and `trig` treated as a single, 13-bit trigger offset (i.e. `Imm` is the higher order bits).

  If the second operand is unsigned, it is treated as an absolute target address and the condition must always be zero to have the jump be taken. If it is signed, it is treated like how `Imm` is treated by the 0- and 1-operand variants to calculate the target address. The condition is treated as with the 1-operand variant (based on the second operand and not `Imm`.)




```
jmp {Label:imm}, {Label:trig}
jmp {Label:trig<>imm}
```

- Call – Perform a function call.

  Call the function at the address given as input.

  The first input is used to find the address to call. If it's an unsigned integer, it is interpreted as an address and called directly. If it's a signed integer, it is interpreted as an offset from the address of the call. Therefore, the offset is added to the address of the call, and the resulting address is then called.

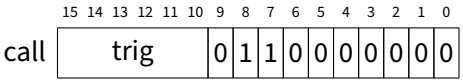
  The call trigger counts as a consuming instruction in the caller's reference counting, and it is this that determines which values become the callee's arguments. Any value whose remaining flight time is 0 when the trigger fires — i.e. whose reference resolves exactly to the trigger — becomes an argument: all such values are delivered, in their ready-queue order, to the first instruction executed in the callee. Any value with remaining flight time greater than 0 is not an argument; the trigger consumes one step of its flight time like any executed instruction, after which its flight time pauses for the duration of the call (see the discussion at the start of this section).

  For example, consider an `add` that is the instruction at the trigger address (the last caller instruction executed before the call takes effect). If the `add` has `ref` $= 0$, its output hits

the trigger and becomes an argument, arriving at the first instruction of the callee. If it has `ref` $= 1$, the output skips the callee entirely and is consumed by the first caller instruction executed after the return (i.e. the instruction following the `add` in the binary).

Since all arguments are delivered to the callee's first instruction, at most four values can be passed this way (subsection 1.1.1). By convention, any arguments beyond the fourth are passed on the stack instead (section 3.1).

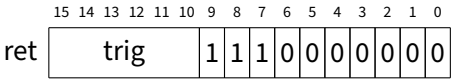
The returned value(s) are sent forward, with the offset starting after the trigger address.




```
call {Label:trig}
```

- Ret – Perform a return from a function call.

  Return arguments are given like how input arguments are given to the call: the return trigger counts as a consuming instruction in the callee, and any value whose reference resolves exactly to the return trigger becomes a return value. Return values are delivered, in their ready-queue order, to the first caller instruction executed after the return (the instruction following the call's trigger address). Like call arguments, at most four values can be returned this way; by convention, any further return values are passed on the stack instead (section 3.1).




```
ret {Label:trig}
```

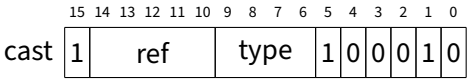
Cast

Change the type of the incoming operands to the given type.

Bit-casts, i.e., does no interpretation of the bits, just changes the type tag.

If the incoming type is larger than the outgoing, the superfluous bytes are dropped.

If the incoming type is smaller than the outgoing type, zero-extension is used if the incoming type is unsigned and sign-extension for signed incoming types.



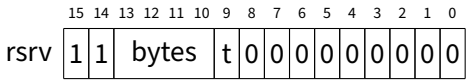

```
cast {Type:type}, {Ref:ref}
```

Stack control

- Reserve – Reserves some number of bytes on the stack.

  If t=1, reserve on the private stack, otherwise on the shared.

  'Bytes' is the power of two amount of bytes to reserve or free. bytes=0 reserves 1 byte, bytes=1 reserves 2 bytes, bytes=2 4 bytes and so on.

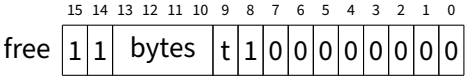
  


  ```
  rsrv {DecPow2:bytes} {{, Private}? {1:t} : {0:t}}
  ```

- Free – Frees some number of bytes on the stack.

  If t=1, free on the private stack, otherwise on the shared.

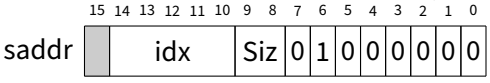
  The number of bytes is calculated like with reserve


  ```
  free {DecPow2:bytes} {{, Private}? {1:t} : {0:t}}
  ```

- SADDR – Get address of stack location

  Returns the address of a stack object. The address is calculated similarly to the load instruction, except signedness not being relevant. The type argument in assembly is used only to obtain the scale (size) of the type, which is encoded in `Siz` as a power-of-two exponent: `Siz` $= 0$ for a scale of 1 byte, 1 for 2 bytes, 2 for 4 bytes, and 3 for 8 bytes.


  ```
  saddr {Type:Siz}[{Dec:idx}]
  ```

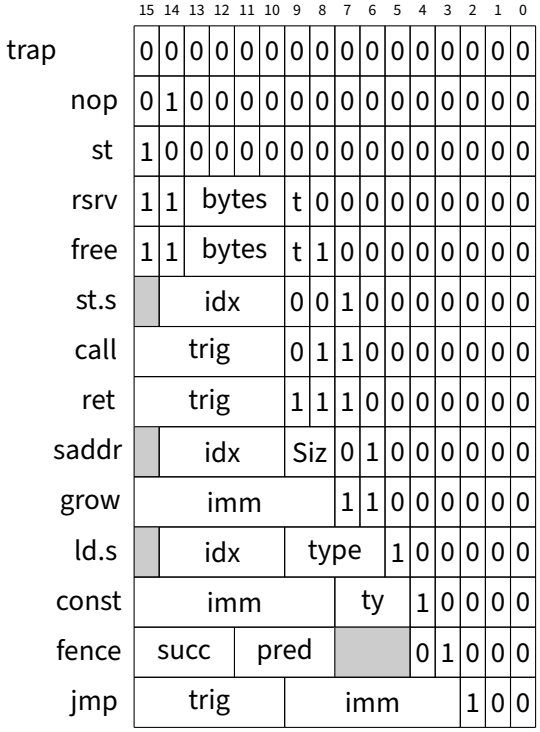
Maybe reserve the lows (e.g. 0 and 1) to do something else useful.

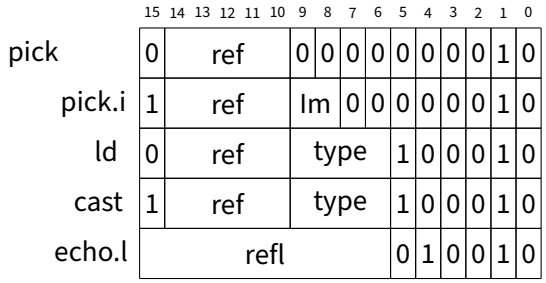
What if given inputs?

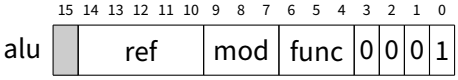
This luckily does not affect in-flight stack-loads. They can be tracked as being relative to the stack base, which does not change regardless of RSRV or FREE.

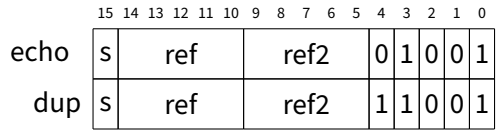
Need some way to align the base, otherwise it becomes impossible to know if indices of various data sizes overlap. Options: * Reserves can take additional alignment operand. * Reserve could

4.6 Encoding Overview









5 Assembly

5.1 ALU Instructions

5.1.1 Different ALU reference modes

```
add Low, High, =>4 # LowFirst
add High, Low, =>4 # HighFirst
add High, =>, Low, =>4 # HighNext
add Low, =>, High, =>4 # LowNext
add Low, =>4 # LowOnly
add High, =>4 # HighOnly
```